The operation is a release operation on the specified memory location. This therefore synchronizes-with an acquire operation on the same memory location that reads the stored value.

Std::Memory_Order_Acquire

The operation is an acquire operation on the specified memory location. If the stored value was written by a release operation, that store synchronizes-with this operation.

Std::Memory_Order_Acq_Rel

The operation must be a read-modify-write operation, and it behaves as both `std::memory_order_acquire` and `std::memory_order_release` on the specified location.

Std::Memory_Order_Seq_Cst

The operation forms part of the single global total order of sequentially consistent operations. In addition, if it's a store, it behaves like a `std::memory_order_release` operation; if it's a load, it behaves like a `std::memory_order_acquire` operation; and if it's a read-modify-write operation, it behaves as both `std::memory_order_acquire` and `std::memory_order_release`. *This is the default for all operations.*

Std::Memory_Order_Consume

The operation is a consume operation on the specified memory location.

D.3.5. std::atomic_thread_fence function

The `std::atomic_thread_fence()` function inserts a "memory barrier" or "fence" in the code to force memory-ordering constraints between operations.

Declaration